solution, the organization may need to take over some extra responsibilities, such as handling security in the cloud, which in turn requires specific skills. There are also other considerations, like business interruptions during the migration, retraining end users, and updating documentation and support materials.

1.6 Cloud native topologies

My explanation of cloud native didn't involve specific technologies or architectures. The CNCF mentions some in its definition, like containers and microservices, but those are just examples. You don't have to use Docker containers for your applications to be cloud native. Think about serverless or PaaS solutions. Writing functions for the AWS Lambda platform or deploying applications to Heroku doesn't require you to build containers. Still, they are categorized as cloud native.

In this section, I'll describe some common cloud native topologies (see figure 1.9). First, I'll introduce the concepts of containers and orchestration, which will be further explored later when I discuss Docker and Kubernetes. Then I'll present the topic of serverless technologies and functions (FaaS). I will not focus much on the FaaS model in this book, but I will cover the basics of how to build serverless applications using Spring Native and Spring Cloud Function.